Diseño y Desarrollo de Sistemas de Información

Mapeo Objeto Relacional (JPA - *Hibernate*)

Curso 2026-27

Objetivos

- Aprender a utilizar una implementación de una herramienta de Mapeo Objeto-Relacional (ORM)
- Aprender a implementar operaciones CRUD en Java utilizando una herramienta de ORM

Mapeo Objeto Relacional. Conceptos y definiciones

- Un ORM es un modelo de programación que permite "mapear" las tablas de una **base de datos relacional** (Oracle, MariaDB, etc.) con **clases** y **objetos** de un lenguaje de programación

- Este mapeo crea una **base de datos virtual** en el entorno del ORM, donde cada tabla se representa como una entidad lógica

- Las operaciones CRUD (Create, Read, Update, Delete) se realizan de forma indirecta y transparente, delegando la interacción con la base de datos en el ORM

- Además de generar código automáticamente, suelen incorporar un lenguaje propio para consultas y gestión de la persistencia de los datos

- Los objetos o entidades de la BD virtual creada por el ORM se gestionan usando lenguajes de propósito general (por ejemplo, Java)

- La interacción con el SGBD se realiza mediante los métodos propios del ORM

- Trabajar directamente con las entidades del modelo, sin escribir SQL, aporta ventajas clave:

  - Acelera el desarrollo
  - Simplifica la implementación
  - Reduce errores por SQL manual

Ventajas e inconvenientes de los ORM

- Ventajas
  - Menor necesidad de escribir código SQL (más sencillo para quienes no lo dominan)
  - Mayor reutilización y mantenimiento del código
  - Reducción del tiempo de desarrollo
  - Seguridad: evita inyecciones SQL y errores comunes
  - Están bien optimizados para las operaciones CRUD *insert*, *delete* y *update*, aunque algo menos para las operaciones de recuperación (*select*)

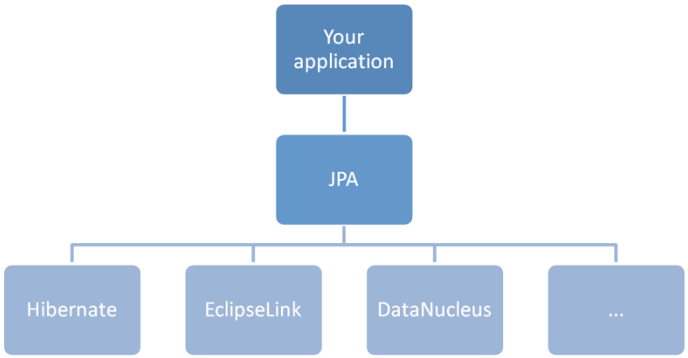
- Inconvenientes
  - En entornos con gran carga puede reducir el rendimiento, ya que se está agregando una capa extra al sistema
  - La curva aprendizaje puede ser eleveda
  - Si las consultas son complejas, el ORM no garantiza una buena optimización, tal y como lo realizaría un desarrollador de forma nativa. De ahí que las implementaciones de ORM ofrezcan extensiones para escribir consultas en SQL nativo

¿Qué es JPA? ¿Qué es Hibernate?

- JPA (*Java Persistence API*) es la especificación que define el funcionamiento de la persistencia de objetos en Java. Por ejemplo, establece las anotaciones que se deben usar, cómo se deben persistir los objetos, etc.

- JPA es un documento, no una implementación. Para poder trabajar con esta API se necesita un *framework* que implemente la especificación

- *Hibernate* es uno de los *frameworks* que implementa la especificación JPA. Hay otros como *EclipseLink*, *DataNucleus*, *TopLink*, etc.



Diferencia entre JDBC e Hibernate

```
...
ps = conexion.getConexion().prepareStatement(
        "INSERT INTO CLIENTE " +
        "(idCliente, nombre, apellidos, telefono, fechaNacimiento, " +
        "email, idiomaPreferido, categoria) " +
        "VALUES (?, ?, ?, ?, ?, ?, ?, ?)" );

ps.setString(1, cliente.getIdCliente());
ps.setString(2, cliente.getNombre());
ps.setString(3, cliente.getApellidos());
ps.setString(4, cliente.getTelefono());
ps.setDate(5, java.sql.Date.valueOf(cliente.getFechaNacimiento()));
ps.setString(6, cliente.getEmail());
ps.setString(7, cliente.getIdiomaPreferido());
ps.setString(8, cliente.getCategoria());

ps.executeUpdate();
ps.close();
...
```

```
...
Session sesion = HibernateUtil.getSessionFactory().openSession();
Transaction transaccion = sesion.beginTransaction();

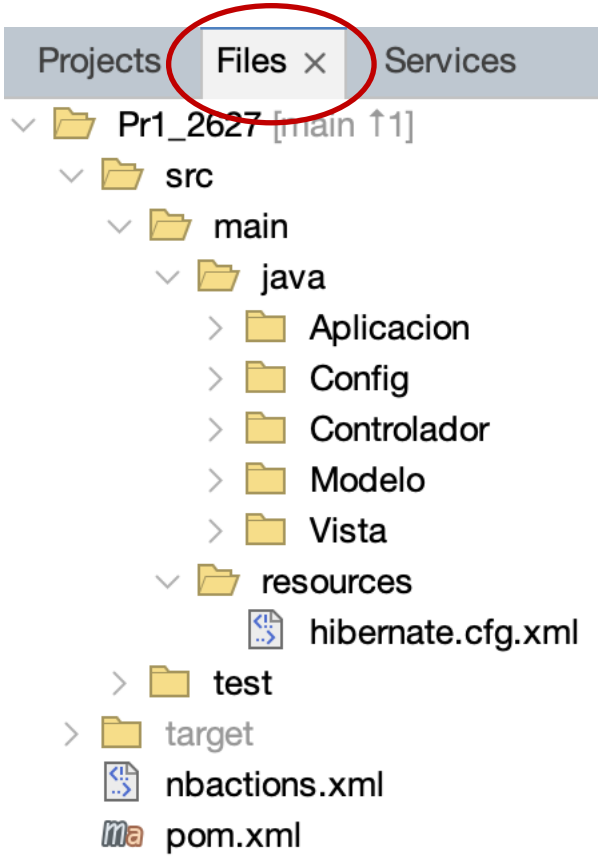
sesion.persist(cliente);

transaccion.commit();

sesion.close();
...
```

Fichero de configuración de Hibernate

- Una forma habitual de configurar Hibernate es mediante el fichero hibernate.cfg.xml
- En un proyecto Maven se sitúa en src/main/resources, de modo que quede disponible en la raíz del *classpath*



- Si no existe la carpeta *resources*, deberá crearse
- Una vez en la carpeta, con el botón derecho, seleccionar "Nuevo" + "Fichero xml"

Fichero de configuración de Hibernate

MariaDB

```
<?xml version="1.0" encoding="utf-8"?>
<!DOCTYPE hibernate-configuration PUBLIC "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
"http://www.hibernate.org/dtd/hibernate-configuration">
<hibernate-configuration>
        <session-factory>
                <property name="hibernate.connection.driver_class">org.mariadb.jdbc.Driver</property>
                <property name="hibernate.connection.url">jdbc:mariadb://172.18.1.241:3306/BaseDeDatos*</property>
                <property name="hibernate.connection.username">usuario</property>
                <property name="hibernate.connection.password">contraseña</property>
                <property name="hibernate. show_sql">true</property>
                <property name="hibernate.format_sql">true</property>
</session-factory>
</hibernate-configuration>
```

BaseDeDatos* -> nombre de la base de datos a la que se conecta la aplicación (por ejemplo, DDSI_001)

Conexión y comunicación entre la aplicación y la base de datos

- La conexión y la comunicación con la base de datos se gestiona, principalmente, con:

  - Clase *SessionFactory* -> fábrica de sesiones. Se crea, normalmente, una única vez para la aplicación
  - Clase *Session* -> representa una unidad de trabajo con la base de datos. Es de corta duración y mantiene el contexto de persistencia de los objetos
  - Clase *Transaction* -> delimita una transacción de base de datos (commit, rollback)

- La creación y gestión de *SessionFactory* suele encapsularse en una clase auxiliar propia que llamaremos *HibernateUtil.java*, y que ubicaremos en un paquete del proyecto (por ejemplo, *Config*)

Ejemplo de *HibernateUtil.java*

```
import org.hibernate.SessionFactory;
import org.hibernate.boot.Metadata;
import org.hibernate.boot.MetadataSources;
import org.hibernate.boot.registry.StandardServiceRegistry;
import org.hibernate.boot.registry.StandardServiceRegistryBuilder;

public class HibernateUtil {

   // Se crea una única instancia de SessionFactory al cargar la clase
   private static final SessionFactory sessionFactory = buildSessionFactory();

   private static SessionFactory buildSessionFactory() {
      try {
          StandardServiceRegistry serviceRegistry = new StandardServiceRegistryBuilder()
                .configure("hibernate.cfg.xml").build();

         Metadata metadata = new MetadataSources(serviceRegistry).getMetadataBuilder().build();

         // Devuelve la fábrica de sesiones
         return metadata.getSessionFactoryBuilder().build();

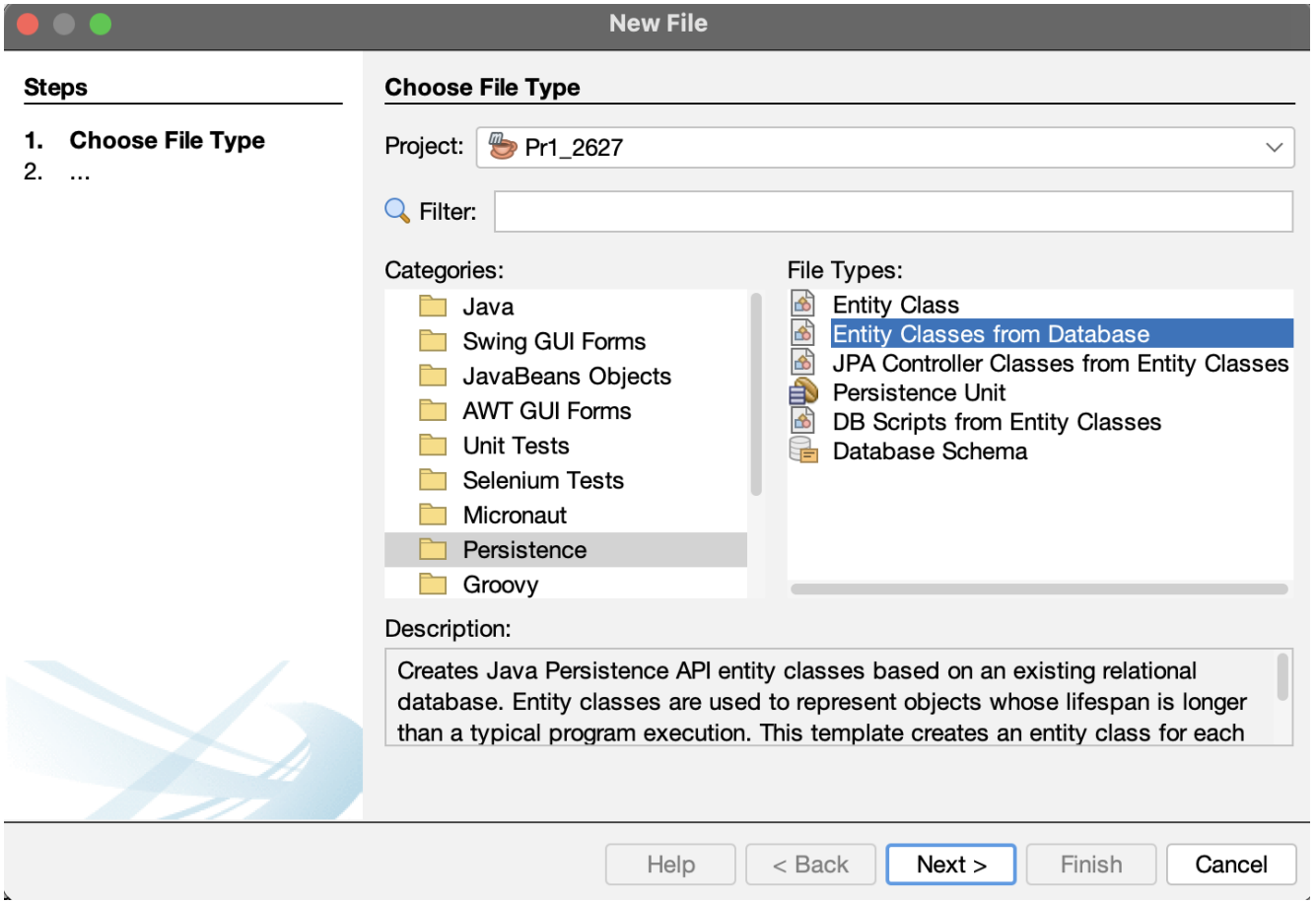
      } catch (Exception ex) {
          System.err.println ("Build SessionFactory failed :" + ex);
          throw new ExceptionInInitializerError(ex);  }
   }

   public static SessionFactory getSessionFactory() {  return sessionFactory;   }

   public static void close() {
      if ((sessionFactory != null) && (sessionFactory.isClosed() == false))  sessionFactory.close();
   }
}
```

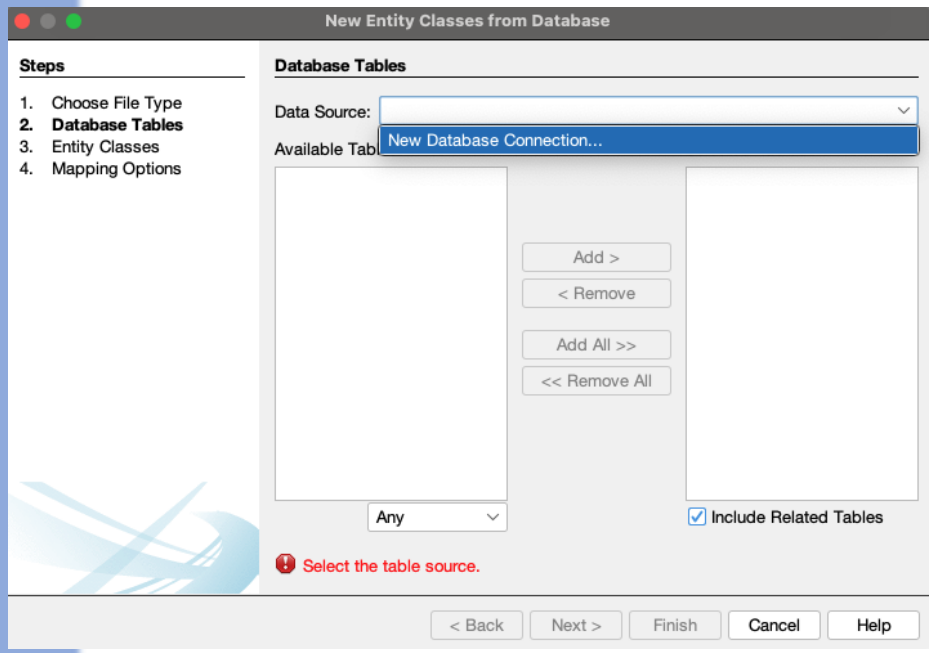
Creación de las clases derivadas de las tablas de la BD

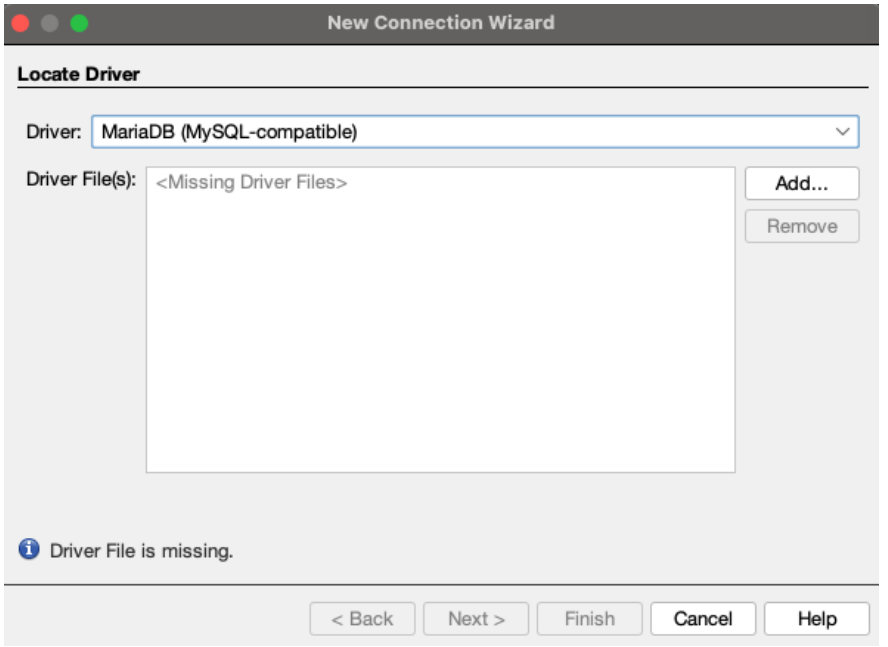
- El siguiente paso consiste en crear las clases Java sobre las que se mapearán las tablas de la base de datos
- Puede hacerse de forma manual o con herramientas del IDE. En este caso, nos apoyaremos en una funcionalidad de **NetBeans** llamada "Entity classes from databases", que se encuentra dentro del módulo "Persistence"



Creación de las clases derivadas de las tablas de la BD

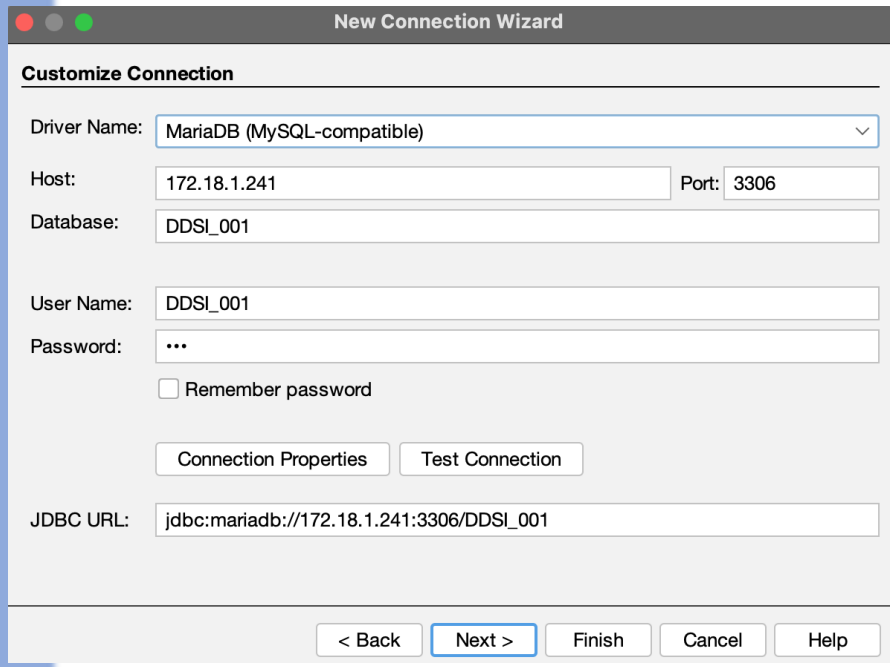
- Para mapear las tablas debemos elegir la conexión al servidor de base de datos
- Si en *Netbeans* aún no hemos creado ninguna conexión, se puede crear en este paso. Por ejemplo, para MariaDB

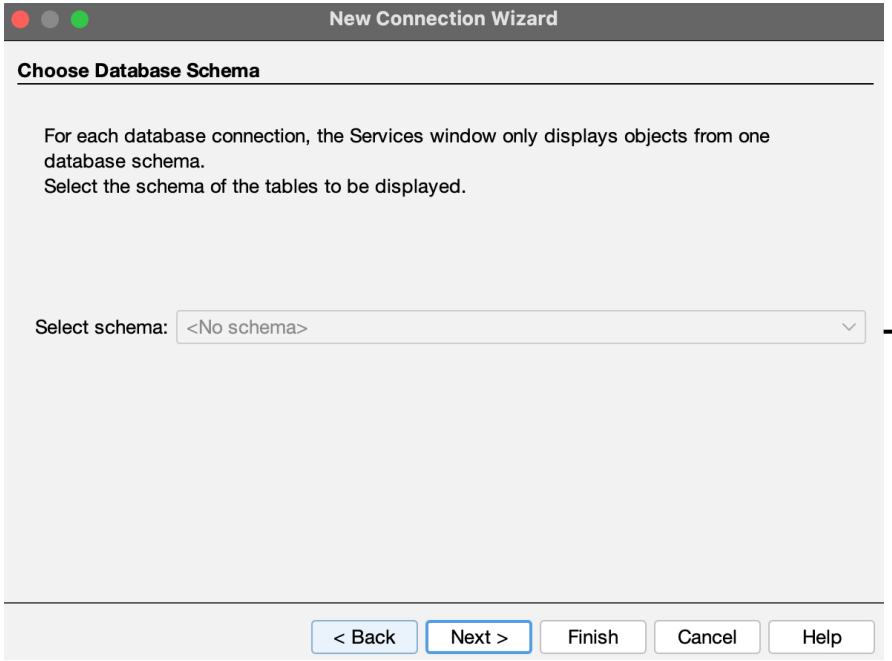




Debéis añadir el driver de MariaDB (**mariadb-java-client-3.5.10.jar**)

Creación de las clases derivadas de las tablas de la BD





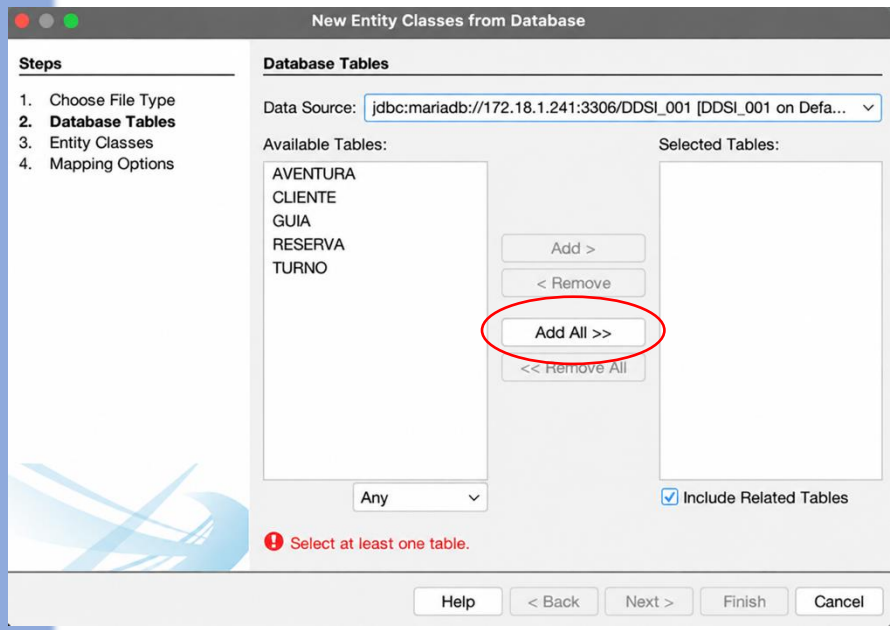
En MariaDB no hay que elegir "schema" puesto que tomará el del usuario/password seleccionado en el paso anterior

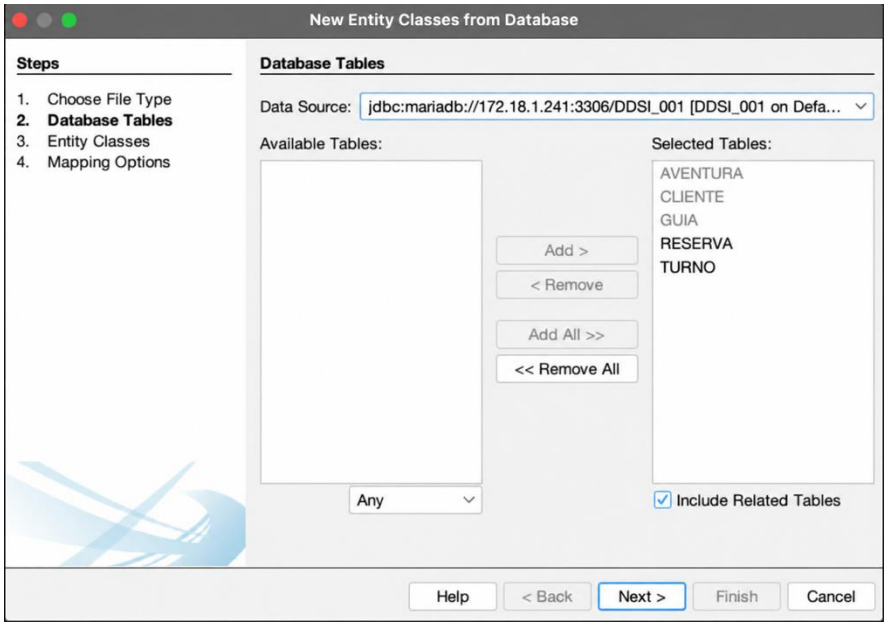
Creación de las clases derivadas de las tablas de la BD

IMPORTANTE

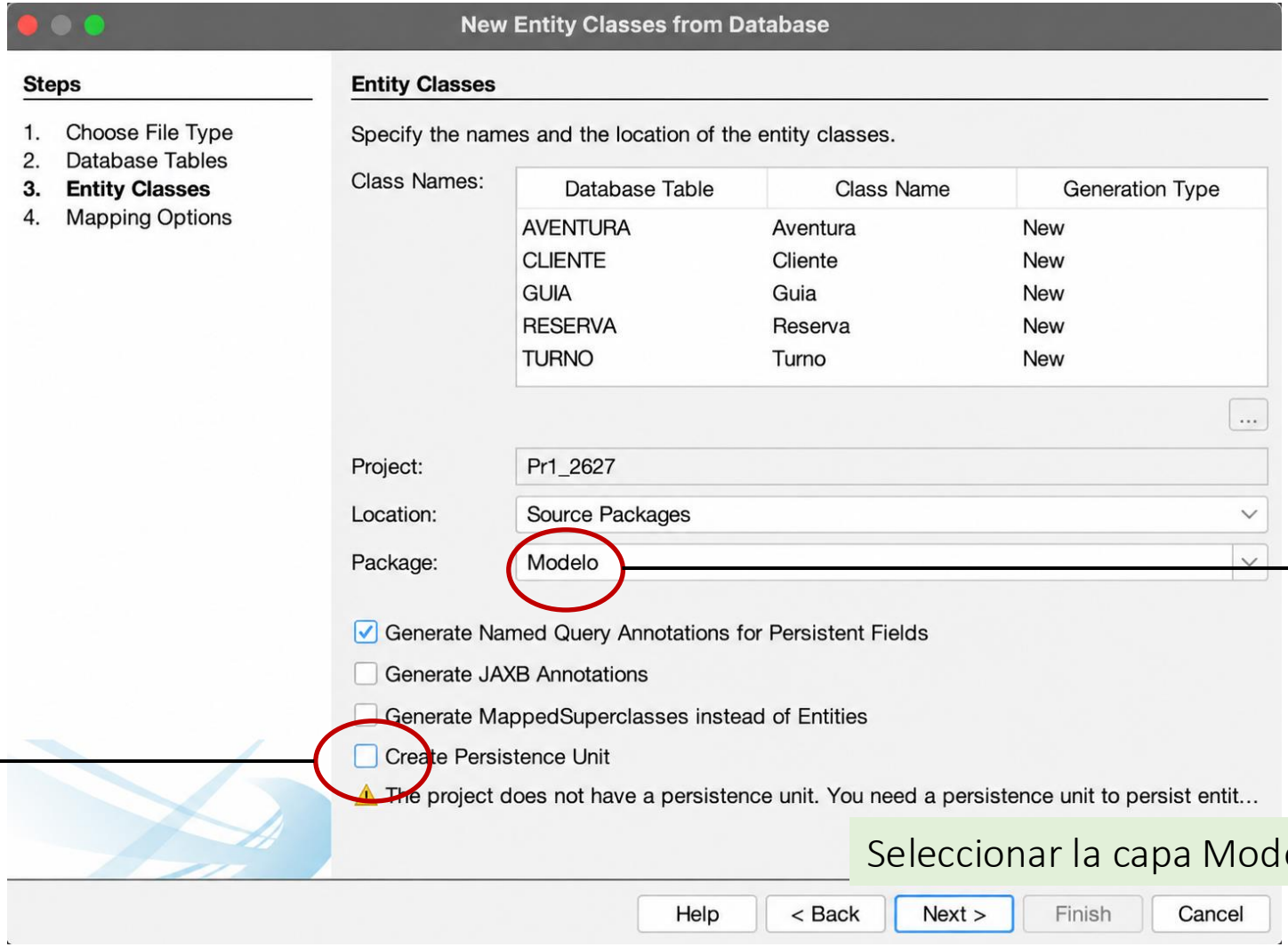
- En JPA/Hibernate, las tablas intermedias de relaciones muchos-a-muchos sin atributos propios pueden mapearse directamente mediante @ManyToMany, sin crear una entidad específica para la tabla intermedia

- Si la relación tiene atributos propios, la tabla intermedia debe modelarse como una entidad





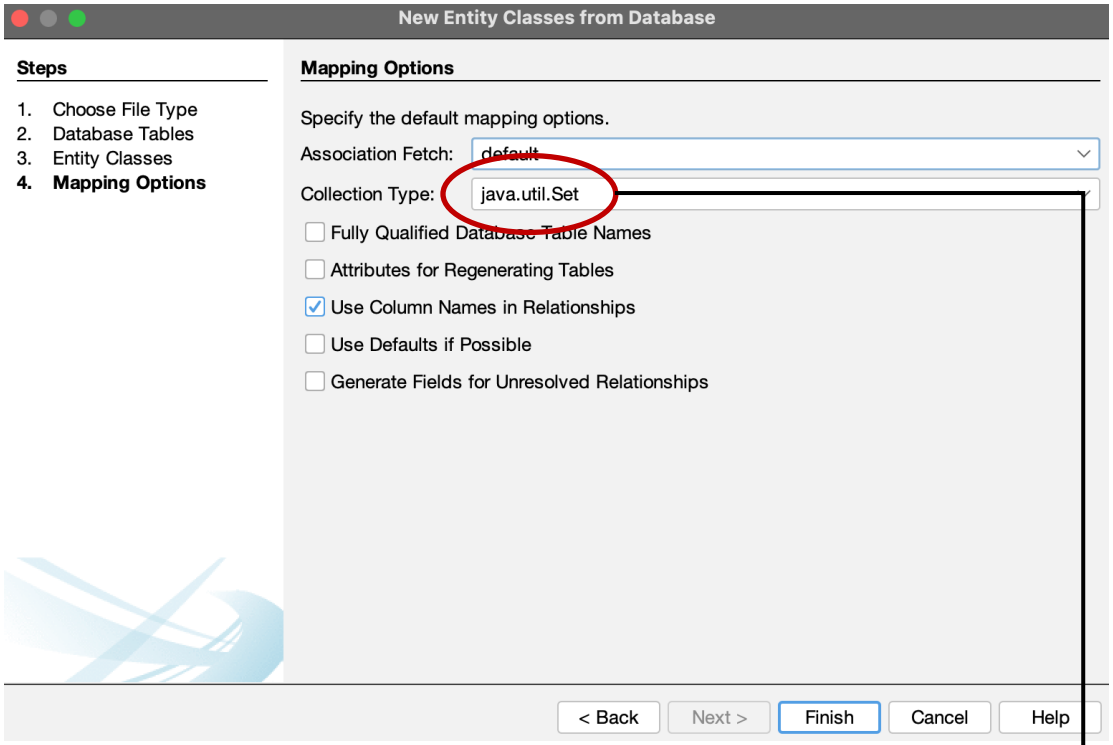
Creación de las clases derivadas de las tablas de la BD



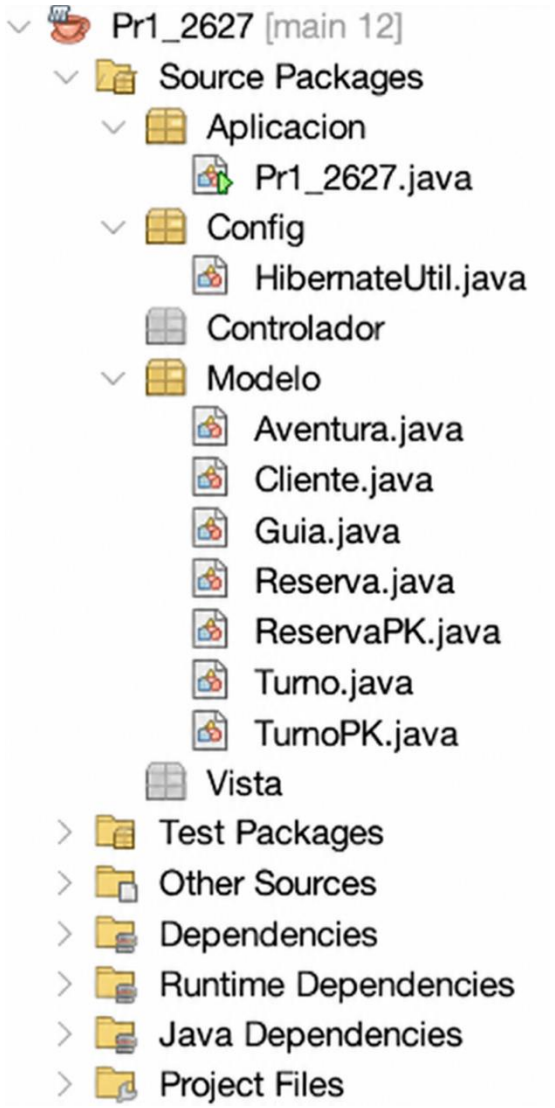
Seleccionar la capa Modelo

Desmarcar "*Create Persistence Util*". En nuestro proyecto usaremos *hibernate.cfg.xml* y *HibernateUtil*

Creación de las clases derivadas de las tablas de la BD



Seleccionar ***java.util.Set*** como tipo de colección para almacenar las colecciones de valores



Algunas anotaciones

- @Entity. Indica que la clase es una entidad persistente
- @Table. Especifica la tabla de la BD asociada a la entidad
- @Column. Permite especificar características de la columna asociada al atributo: nombre, nulabilidad, unicidad, longitud, etc
- @Id. Indica el atributo que constituye la clave primaria
- @Basic. Indica un mapeo básico de un atributo persistente. Su uso normalmente es opcional

- @NamedQuery. Define una consulta JPQL con nombre para poder reutilizarla desde el código

- @OneToMany. Representa una asociación "uno a muchos"
- @ManyToOne. Representa una asociación "muchos a uno", normalmente asociada a una clave ajena

- @JoinColumn. Especifica la columna que se utiliza para realizar la asociación entre dos entidades, normalmente una clave ajena. Permite indicar, entre otras cosas, el nombre de la columna y la columna referenciada

Clase Cliente.java (1/2)

```
@Entity
@Table(name = "CLIENTE")
@NamedQueries({
   @NamedQuery(name = "Cliente.findAll", query = "SELECT c FROM Cliente c"), … })

public class Cliente implements Serializable {
   private static final long serialVersionUID = 1L;
   @Id
   @Basic(optional = false)
   @Column(name = "idCliente")
   private String idCliente;
   @Basic(optional = false)
   @Column(name = "nombre")
   private String nombre;
   …
   @Basic(optional = false)
   @Column(name = "fechaNacimiento")
   private LocalDate fechaNacimiento;
   @Column(name = "email")
   private String email;
   @Basic(optional = false)
   @Column(name = "idiomaPreferido")
   private String idiomaPreferido;
   @Basic(optional = false)
   @Column(name = "categoria")
   private String categoria;
   @OneToMany(cascade = CascadeType.ALL, mappedBy = "cliente")
   private Set<Reserva> reservaSet;
```

(1) **@OneToMany(cascade = CascadeType.ALL, mappedBy = "cliente")** → eliminamos "cascade = CascadeType.ALL" para que el control lo tenga la política definida en las claves ajenas de la BD

(2) **mappedBy = "cliente"** → la relación se gestiona desde el atributo "cliente" de la clase Reserva.java

(3) Inicializar esta estructura y ponerle un nombre más significativo ("reservas")
private Set<Reserva> reservas = new HashSet<Reserva>();

Clase Cliente.java (2/2)

```
public Cliente() { }

 public Cliente(String idCliente) { this.idCliente = idCliente; }

public Cliente(String idCliente, String nombre, String apellidos, LocalDate fechaNacimiento, String
                        idiomaPreferido, String categoria) {
                                this.idCliente = idCliente;
                                this.nombre = nombre;
                                this.apellidos = apellidos;
                                this.fechaNacimiento = fechaNacimiento;
                                this.idiomaPreferido = idiomaPreferido;
                                this.categoria = categoria;  }

   public String getIdCliente() { return idCliente;  }
   public void setIdCliente(String idCliente) { this.idCliente = idCliente;  }

   public String getNombre() { return nombre; }
   public void setNombre(String nombre) { this.nombre = nombre; }
...
    public Set<Reserva> getReservas() { return reservas; }
    public void setReservas(Set<Reserva> reservas) { this.reservas = reservas; }

@Override
    // Otros métodos (comparación, listado, etc.)
```

Usad los mismos nombres

- Se puede comprobar que la herramienta ha generado, de forma automática, 3 constructores para la clase **Cliente**:

```
public Cliente ()
public Cliente (String idCliente)
public Cliente (String idCliente, String nombre, String apellidos, LocalDate fechaNacimiento,
                String idiomaPreferido, String categoria)
```

- Sin embargo, no ha generado el constructor con todos los atributos (ya que algunos no son obligatorios) por lo que, si queremos disponer de un constructor con todos los atributos, tendremos que añadirlo manualmente:

```
public Cliente (String idCliente, String nombre, String apellidos, String telefono, LocalDate
                fechaNacimiento, String email, String idiomaPreferido, String categoria)
```

Clase Aventura.java (1/2)

```
@Entity
@Table(name = "AVENTURA")
@NamedQueries({
   @NamedQuery(name = "Aventura.findAll", query = "SELECT a FROM Aventura a"), ...})

public class Aventura implements Serializable {
   private static final long serialVersionUID = 1L;
   @Id
   @Basic(optional = false)
   @Column(name = "idAventura")
   private String idAventura;
   @Basic(optional = false)
   @Column(name = "nombre")
   private String nombre;
   @Lob
   @Column(name = "descripcion")
   private String descripcion;
   ...
   @Basic(optional = false)
   @Column(name = "precio")
   private BigDecimal precio;
   @Basic(optional = false)
   @Column(name = "edadMinima")
   private short edadMinima;

   @OneToMany(cascade = CascadeType.ALL, mappedBy = "aventura")
   private Set<Turno> turnoSet;
```

(1) @OneToMany(cascade = CascadeType.ALL, mappedBy = "aventura") → eliminamos "cascade = CascadeType.ALL" para que el control lo tenga la política definida en las claves ajenas de la BD

(2) mappedBy = "aventura" → la relación se gestiona desde el atributo "idAventura" de la clase Turno.java

(3) Inicializar esta estructura y ponerle un nombre más significativo ("turnos")
private Set<Turno> turnos = new HashSet<Turno>();

Clase Aventura.java (2/2)

```
public Aventura() {  }
public Aventura(String idAventura) { this.idAventura = idAventura; }
public Aventura(String idAventura, String nombre, String tipo, String dificultad, short duracion, BigDecimal
                precio, short edadMinima) {
                        this.idAventura = idAventura;
                        this.nombre = nombre;
                        this.tipo = tipo;
                        this.dificultad = dificultad;
                        this.duracion = duracion;
                        this.precio = precio;
                        this.edadMinima = edadMinima;   }

  public String getIdAventura() { return idAventura; }
  public void setIdAventura(String idAventura) { this.idAventura = idAventura; }
  public String getNombre() {  return nombre; }
  public void setNombre(String nombre) { this.nombre = nombre; }
...
  public Set<Turno> getTurnos() { return turnos;  }
  public void setTurnos(Set<Turno> turnos) { this.turnos = turnos; }


@Override
   // Otros métodos (comparación, listado, etc.)
```

Usad los mismos nombres

Clase Turno.java (1/2)

```
@Entity
@Table(name = "TURNO")
@NamedQueries({ ... })

public class Turno implements Serializable {
   private static final long serialVersionUID = 1L;
   @Id
   @Basic(optional = false)
   @Column(name = "idTurno")
   private String idTurno;

   @Basic(optional = false)
   @Column(name = "fechaHora")
   private LocalDateTime fechaHora;
   @Basic(optional = false)
   @Column(name = "capacidad")
   private short capacidad;

   @JoinColumn(name = "idAventura", referencedColumnName = "idAventura")
   @ManyToOne(optional = false)
   private Aventura aventura;

   @JoinColumn(name = "idGuia", referencedColumnName = "idGuia")
   @ManyToOne(optional = false)
   private Guia guia;

   @OneToMany(mappedBy = "turno")
   private Set<Reserva> reservaSet;
```

Atributo "idTurno" que hemos usado como clave primaria sustituta simple

El atributo "fechaHora" no forma parte del identificador

(1) @JoinColumn usa idAventura como Clave Ajena hacia AVENTURA
(2) @ManyToOne representa que muchos turnos pueden pertenecer a una misma aventura
(3) Renombramos el atributo generado **idAventura** como **aventura**, ya que contiene un objeto Aventura, no un identificador. Este es el atributo al que hará referencia **mappedBy="aventura"** en Aventura.java.

(1) @ManyToOne indica que varios turnos pueden tener el mismo guía
(2) @JoinColumn usa idGuia como Clave Ajena hacia GUIA
(3) Renombramos el atributo generado **idGuia** como **guia**, ya que contiene un objeto Guia. Este es el atributo al que hará referencia **mappedBy="guia"** en Guia.java.

(1) @OneToMany(mappedBy="turno") representa las reservas asociadas a un turno. **mappedBy="turno"** indica que el lado propietario de la relación es el atributo turno de Reserva.java.
(2) Eliminamos "cascade = CascadeType.ALL" e inicializamos y renombramos el conjunto.
private Set<Reserva> reservas = new HashSet<Reserva>();

Clase Turno.java (2/2)

```
public Turno() {  }
public Turno(String idTurno) { this.idTurno = idTurno; }
public Turno(String idTurno, LocalDateTime fechaHora, short capacidad) {
      this.idTurno = idTurno;
      this.fechaHora = fechaHora;
      this.capacidad = capacidad;   }
public Turno(String idTurno, LocalDateTime fechaHora, short capacidad, Aventura aventura, Guia guia) {
      this.idTurno = idTurno;
      this.fechaHora = fechaHora;
      this.capacidad = capacidad;
      this.aventura = aventura;
      this.guia = guia;   }

   public String getIdTurno() { return idTurno; }
   public void setIdTurno(String idTurno) { this.idTurno = idTurno;  }
   public LocalDateTime getFechaHora() { return fechaHora;  }
   public void setFechaHora(LocalDateTime fechaHora) { this.fechaHora = fechaHora; }
...
   public Aventura getAventura() {  return aventura;  }
   public void setAventura(Aventura aventura) {  this.aventura = aventura;  }
   public Guia getGuia() { return guia;  }
   public void setGuia(Guia guia) { this.guia = guia; }
   public Set<Reserva> getReservas() {  return reservas; }
   public void setReservas(Set<Reserva> reservas) { this.reservas = reservas; }

@Override
   // Otros métodos (comparación, listado, etc.)
```

Constructor añadido

Renombramos los métodos get() y set() para que sean coherentes con los nuevos nombres de los atributos: aventura y guia

Clase Reserva.java (1/2)

```
@Entity
@Table(name = "RESERVA")
@NamedQueries({ ... })

public class Reserva implements Serializable {
   private static final long serialVersionUID = 1L;
   @Id
   @Basic(optional = false)
   @Column(name = "idReserva")
   private String idReserva;
   @Basic(optional = false)
   @Column(name = "fechaReserva")
   private LocalDateTime fechaReserva;
   ...

   @JoinColumn(name = "idCliente", referencedColumnName = "idCliente")
   @ManyToOne(optional = false)
   private Cliente cliente;

   @JoinColumn(name = "idTurno", referencedColumnName = "idTurno")
   @ManyToOne(optional = false)
   private Turno turno;
```

Atributo "idReserva" que hemos usado como **clave primaria sustituta simple**

(1) @ManyToOne indica que cada reserva está asociada a un único cliente y que un cliente puede tener varias reservas
(2) @JoinColumn mapea la Clave Ajena idCliente hacia CLIENTE
(3) Renombramos el atributo generado **idCliente** como **cliente**, ya que contiene un objeto Cliente, no un identificador. Este es el atributo al que hará referencia **mappedBy="cliente"** en Cliente.java

(1) @ManyToOne indica que cada reserva está asociada a un único turno y que un turno puede tener varias reservas
(2) @JoinColumn mapea la Clave Ajena idTurno hacia TURNO
(3) Renombramos el atributo generado **idTurno** como **turno**, ya que contiene un objeto Turno, no un identificador. Este es el atributo al que hará referencia **mappedBy="turno"** en Turno.java

Clase Reserva.java (2/2)

```
public Reserva() {  }
public Reserva(String idReserva) { this.idReserva = idReserva;  }
public Reserva(String idReserva, LocalDateTime fechaReserva, BigDecimal importe, String estado) {
      this.idReserva = idReserva;
      this.fechaReserva = fechaReserva;
      this.importe = importe;
      this.estado = estado;  }

  public Reserva(String idReserva, LocalDateTime fechaReserva, BigDecimal importe, String estado,
        Short valoracion, Cliente cliente, Turno turno) {
      this.idReserva = idReserva;
      this.fechaReserva = fechaReserva;
      this.importe = importe;
      this.estado = estado;
      this.valoracion = valoracion;
      this.cliente = cliente;
      this.turno = turno;  }

public String getIdReserva() {  return idReserva;  }
public void setIdReserva(String idReserva) {  this.idReserva = idReserva; }
...
public Cliente getCliente() { return cliente; }
public void setCliente(Cliente cliente) { this.cliente = cliente; }
public Turno getTurno() { return turno;  }
public void setTurno(Turno turno) { this.turno = turno;  }

@Override
   // Otros métodos (comparación, listado, etc.)
```

Constructor añadido

Renombramos los métodos get() y set() para que sean coherentes con los nuevos nombres de los atributos: cliente y turno

Clase Guia.java (1/2)

```
@Entity
@Table(name = "GUIA")
@NamedQueries({ ... })

public class Guia implements Serializable {
   private static final long serialVersionUID = 1L;
   @Id
   @Basic(optional = false)
   @Column(name = "idGuia")
   private String idGuia;
   @Basic(optional = false)
   @Column(name = "nombre")
   private String nombre;
   ...
   @Basic(optional = false)
   @Column(name = "fechaAlta")
   private LocalDate fechaAlta;
   ...

   @OneToMany(mappedBy = "guia")
   private Set<Turno> turnoSet;
```

(1) @OneToMany(mappedBy="guia") representa los turnos asociados a un guía. **mappedBy="guia"** indica que el lado propietario de la relación es el atributo guia de Turno.java.

(2) Eliminamos "cascade = CascadeType.ALL" e inicializamos y renombramos el conjunto.

```
private Set<Turno> turnos = new HashSet<Turno>();
```

Clase Guia.java (2/2)

```
public Guia() {  }
public Guia(String idGuia) { this.idGuia = idGuia; }
public Guia(String idGuia, String nombre, String apellidos, String telefono, String email, LocalDate fechaAlta) {
        this.idGuia = idGuia;
        this.nombre = nombre;
        this.apellidos = apellidos;
        this.telefono = telefono;
        this.email = email;
        this.fechaAlta = fechaAlta;  }


public String getIdGuia() { return idGuia; }
public void setIdGuia(String idGuia) { this.idGuia = idGuia; }
public String getNombre() { return nombre; }
public void setNombre(String nombre) { this.nombre = nombre; }
...
public LocalDate getFechaAlta() { return fechaAlta; }
public void setFechaAlta(LocalDate fechaAlta) { this.fechaAlta = fechaAlta; }
public Set<Turno> getTurnos() { return turnos;  }
public void setTurnos(Set<Turno> turnos) { this.turnos = turnos; }


 @Override
    // Otros métodos (comparación, listado, etc.)
```

Renombramos los métodos get() y set() para que sean coherentes con el nuevo nombre del atributo turnos

Modificación del fichero hibernate.cfg.xml

- Una vez realizado el mapeo de la base de datos, es necesario editar el fichero de configuración hibernate.cfg.xml para añadir las etiquetas <mapping class>

```
<?xml version="1.0" encoding="utf-8"?>
<!DOCTYPE hibernate-configuration PUBLIC "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
"http://www.hibernate.org/dtd/hibernate-configuration">
<hibernate-configuration>
        <session-factory>
                <property name="hibernate.connection.driver_class">org.mariadb.jdbc.Driver</property>
                <property name="hibernate.connection.url">jdbc:mariadb://172.18.1.241:3306/BaseDeDatos*</property>
                <property name="hibernate.connection.username">usuario</property>
                <property name="hibernate.connection.password">contraseña</property>
                <property name="hibernate.show_sql">true</property>
                 <!-- Entidades -->
                <mapping class="Modelo.Cliente"/>
                <mapping class="Modelo.Guia"/>
                <mapping class="Modelo.Aventura"/>
                <mapping class="Modelo.Turno"/>
                <mapping class="Modelo.Reserva"/>
        </session-factory>
</hibernate-configuration>
```

- Las etiquetas <mapping class> se usan para definir las clases que se mapean con las tablas de la base de datos

Inicialización de Hibernate y SessionFactory

- La comunicación con la base de datos se gestiona a partir de un objeto SessionFactory, que constituye el punto de entrada de Hibernate para crear sesiones
- La aplicación obtiene la instancia de SessionFactory mediante el método HibernateUtil.getSessionFactory()
- Generalmente se crea una única SessionFactory para toda la aplicación y se mantiene activa hasta el cierre del programa. A partir de ella se crearán objetos Session para realizar las operaciones con la BD
- La configuración necesaria para crear SessionFactory se obtiene del fichero hibernate.cfg.xml, situado en src/main/resources

```
try {
        // Se obtiene la SessionFactory definida en HibernateUtil
        SessionFactory sessionFactory = HibernateUtil.getSessionFactory();
        System.out.println("Hibernate inicializado correctamente");

} catch (ExceptionInInitializerError e) {
        Throwable cause = e.getCause();
        System.out.println("Error al inicializar Hibernate. Revise el fichero .cfg.xml: " + cause.getMessage());
}
```

Esquema general de una operación con Hibernate

- Esquema general de funcionamiento para realizar una operación a la BD

```
Session sesion = null;
Transaction tr = null;

try {
        sesion = sessionFactory.openSession();
        tr = sesion.beginTransaction();
        Cliente cliente = sesion.find (Cliente.class, "CL001");
        System.out.println (cliente.getNombre());
        tr.commit();

} catch (Exception e) {
        if (tr != null && tr.isActive()) tr.rollback();
        System.out.println ("Error en la recuperación " +
                                        e.getMessage());
} finally {
        if (sesion != null && sesion.isOpen()) {
                sesion.close();
        }
}
```

Asumimos que **sessionFactory** ya está inicializada correctamente

Se abre la sesión mediante el método **openSession()** de la clase **SessionFactory**

Se comienza la transacción mediante el método **beginTransaction()** de la clase **Session**

Se realiza la operación CRUD. En este caso, se recupera un objeto de la clase **Cliente** (una fila de la tabla **CLIENTE**) y se muestra el nombre del cliente recuperado

Si la operación tiene éxito, se confirma la transacción (**commit**)

Si la operación falla, se deshace la transacción (**rollback**)

Finalmente, en cualquier caso, se cierra la sesión

Las operaciones sobre la base de datos, incluidas las consultas, deben realizarse dentro de una transacción. De esta forma se delimita claramente la unidad de trabajo y se garantiza un comportamiento consistente. Este esquema será la estructura general utilizada para realizar las operaciones CRUD desde los controladores

Operaciones CRUD con Hibernate

- El método find(Class<T> entityClass, Object primaryKey) recupera un objeto asociado a una tabla mediante su clave principal. Devuelve nulo si no existe ninguna tupla con ese valor de clave

```
Cliente cliente = sesion.find(Cliente.class, "CL001");

if (cliente != null) { System.out.println(cliente.getNombre()); }
```

Se recupera en el objeto "cliente" la fila de la tabla CLIENTE cuyo idCliente = "CL001". En este caso, se muestra en pantalla el nombre del cliente recuperado mediante el método getNombre()

Consultas

- *Hibernate* permite realizar consultas mediante su propio lenguaje denominado HQL (*Hibernate Query Language*) o realizarlas directamente en lenguaje SQL ("nativas")

- Existen diversas formas de ejecutar consultas. Los métodos más habituales, y que usaremos en estas prácticas, son createQuery(), createNamedQuery() y createNativeQuery()

- Todos los métodos devuelven objetos de tipo Query o TypedQuery<entityClass>

Operaciones CRUD con Hibernate

```
Query<Cliente> consultaNativa = sesion.createNativeQuery ("SELECT * FROM CLIENTE C
                                WHERE C.email = 'ana.tomia@example.com' ", Cliente.class);
Cliente cliente = (Cliente) consultaNativa.getSingleResult();
System.out.println(cliente.getNombre());
```

Consulta SQL nativa que recupera un único cliente. **Cliente.class** indica que cada fila obtenida debe convertirse en un objeto de la clase **Cliente**. El método **getSingleResult()** se utiliza porque se espera un único resultado

¡¡ Cuidado !! MariaDB es sensible a mayúsculas y minúsculas en el nombre de la tabla cuando se usan consultas nativas. Este nombre debe coincidir con el de la etiqueta **@Table** del POJO correspondiente

La misma consulta con HQL sin usar la cláusula SELECT

```
Query<Cliente> consultaHQL = sesion.createQuery ("FROM Cliente c
                             WHERE c.email = 'ana.tomia@example.com' ", Cliente.class);
```

En HQL se utilizan los nombres de las entidades y de sus atributos Java, no los nombres de las tablas y columnas de la BD

Operaciones CRUD con Hibernate

```
Query<Cliente> consulta = sesion.createNativeQuery ("SELECT * FROM CLIENTE c", Cliente.class);
List<Cliente> clientes = consulta.getResultList();
for (Cliente cliente : clientes)
   System.out.println(cliente.getNombre());
```

Consulta nativa para recuperar todos los clientes

Se utiliza el método **getResultList()** para obtener una lista de resultados. En este ejemplo, se almacena el resultado en una estructura **List** de objetos de tipo **Cliente**, y se recorre mediante un bucle "for" para mostrar el nombre de cada cliente

La misma consulta con HQL

```
Query<Cliente> consulta = sesion.createQuery ("FROM Cliente c", Cliente.class);
```

Operaciones CRUD con Hibernate

La misma consulta con HQL usando una consulta "nombrada"

```
@NamedQuery (name = "Cliente.findAll", query = "SELECT c FROM Cliente c")
```

```
Query <Cliente> consulta = sesion.createNamedQuery ("Cliente.findAll", Cliente.class);
List<Cliente> clientes = consulta.getResultList();
```

La misma consulta con SQL nativo usando una consulta "nombrada"

```
@NamedNativeQueries({
   @NamedNativeQuery(name = "Cliente.TodosNativos", query = "SELECT * FROM CLIENTE C",
                                                    resultClass = Cliente.class) })
```

```
Query<Cliente> consulta = sesion.createNamedQuery ("Cliente.TodosNativos", Cliente.class);
List<Cliente> clientes = consulta.getResultList();
```

Operaciones CRUD con Hibernate

- Por supuesto, también es posible realizar consultas que devuelvan campos específicos

Consulta HQL que devuelve dos atributos de la tabla **AVENTURA**. Como se seleccionan varios atributos, cada resultado se devuelve como un Object[]. Por ello, la colección obtenida será List<Object[]>

```
Query<Object[]> consulta = sesion.createQuery ("SELECT a.nombre, a.precio FROM Aventura a",
                                                Object[].class);
List<Object[]> aventuras = consulta.getResultList();

System.out.println("Nombre\tPrecio");
for (Object[] a : aventuras)
    System.out.println(a[0] + "\t" + a[1]);
```

La consulta SQL nativa es prácticamente equivalente, pero utiliza los nombres de la tabla y de las columnas de la base de datos.

```
Query<Object[]> consulta = sesion.createNativeQuery ("SELECT A.nombre,
                                                A.precio FROM AVENTURA A", Object[].class);
```

Operaciones CRUD con Hibernate

Consulta que devuelve un único campo de la tabla **AVENTURA**

```
Query<String> consulta = sesion.createNativeQuery ("SELECT A.nombre FROM AVENTURA A",
                                                    String.class);
List<String> nombresAventuras = consulta.getResultList();
for (String nombre : nombresAventuras)
        System.out.println(nombre);
```

Cuando la consulta devuelve un único campo, cada elemento del resultado es del tipo correspondiente a ese campo. En este caso, nombre es de tipo **String**, por lo que obtenemos un **List<String>**

Consulta que devuelve un valor único

```
Query<BigDecimal> consulta = sesion.createNativeQuery ("SELECT MAX(A.precio) FROM
                                                        AVENTURA A", BigDecimal.class);

BigDecimal precioMaximo = consulta.getSingleResult();
System.out.println(precioMaximo);
```

La consulta devuelve un único valor: el precio máximo de todas las aventuras. Como sólo se espera un resultado, se utiliza **getSingleResult()**

Operaciones CRUD con Hibernate

Consultas con parámetros

Consulta HQL

```
Query<Guia> consulta = sesion.createQuery( "SELECT g FROM Guia g WHERE g.email = :email",
                                                    Guia.class);
consulta.setParameter ("email", "paco@example.com");

Guia guia = consulta.getSingleResult();
System.out.println(guia.getNombre());
```

:email representa un parámetro de la consulta. Su valor se establece posteriormente mediante **setParameter("email", valor)**. De esta forma, la consulta puede reutilizarse con distintos valores

Una forma más "elegante"

```
Query<Guia> consulta = sesion.createQuery( "SELECT g FROM Guia g WHERE g.email = :email",
            Guia.class).setParameter("email", "paco@example.com");
```

Operaciones CRUD con Hibernate

- Y todavía más... consultas con JOIN

SQL nativo

```
Query<Object[]> consulta = sesion.createNativeQuery("SELECT a.nombre, t.capacidad
                    FROM AVENTURA a INNER JOIN TURNO t ON a.idAventura = t.idAventura",
                    Object[].class);

List<Object[]> resultado = consulta.getResultList();
for (Object[] r : resultado)
   System.out.println(r[0] + "\t" + r[1]);
```

En SQL nativo, el JOIN se realiza entre tablas y debemos indicar la condición **ON**

HQL

```
Query<Object[]> consulta = sesion.createQuery( "SELECT a.nombre, t.capacidad
                    FROM Aventura a JOIN a.turnos t", Object[].class);
```

En HQL, el JOIN se realiza entre entidades utilizando las relaciones definidas en el mapeo de Hibernate